

Especificação de Componentes

Projeto: All Invest · **Atividade:** Especificação de Interfaces e Contratos e Implementação de Componentes (Lab 4)

Integrantes:

- | | |
|---|---|
| 1. Danilo Henrique de Paulo 22.222.008-9 | 4. Pedro Henrique da Fonseca do Nascimento 22.123.099-8 |
| 2. Jônatas da Silva Gonçalves 22.123.095-6 | 5. Vinícius do Nascimento Generoso 22.123.103-8 |
| 3. Kaynã de Deus Ferreira da Silva 22.123.096-4 | 6. Wallace dos Santos Izidoro 22.123.104-6 |

Conforme definido no Lab 3, foram selecionados para implementação o **Investment Component** e o **Portfolio Component**. O Investment requer a interface `IPortfolioService`, fornecida pelo Portfolio. A implementação usa Python 3.12, FastAPI e SQLAlchemy (PostgreSQL/Supabase).

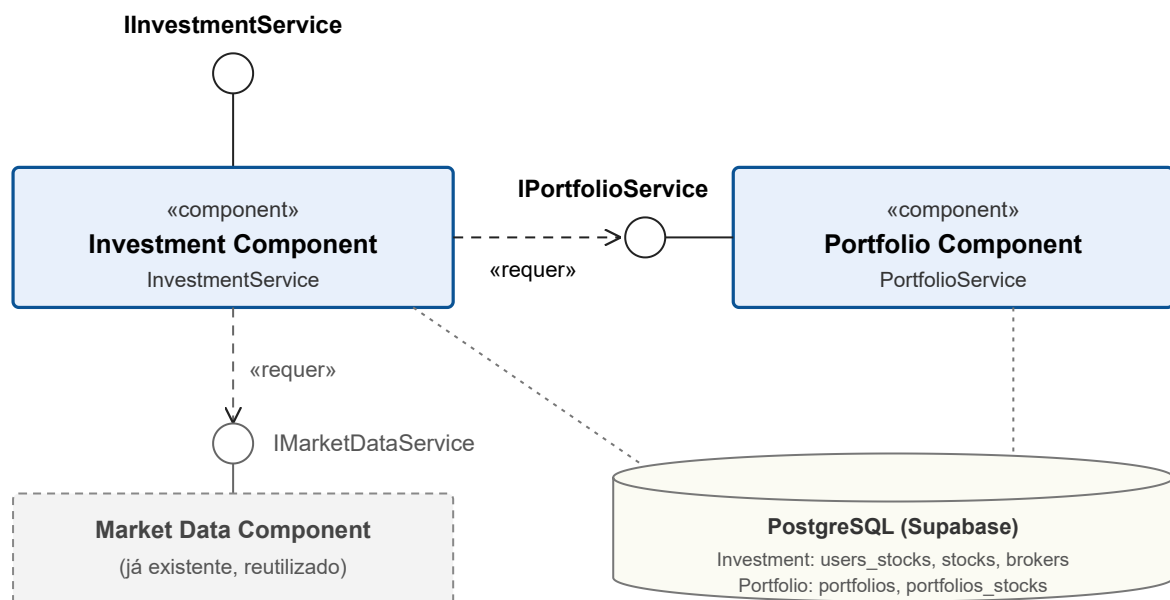


Figura 1 – Recorte da arquitetura do Lab 3 com os dois componentes implementados (em azul).

1. Componente 1

- **Nome:** Investment Component (backend/src/services/investment_service.py, classe InvestmentService)
- **Responsabilidade:** Gerenciar as ações e movimentações do usuário: registrar compras e vendas, consolidar as posições (quantidade e preço médio), associar ações às carteiras e remover investimentos.
- **Operações:**

Operação	Descrição
create_trade	Registra uma compra ou venda (cadastrarAcao / registrarMovimentacao). Valida o ticker na B3, a corretora, o saldo em vendas e, se informada, a carteira; ao final associa o ativo à carteira.
list_trades	Consulta o histórico de operações do usuário (consultarInvestimentos).
get_positions	Consolida as posições por ativo: quantidade, preço médio e carteira associada; aceita filtro por carteira (consultarInvestimentos).
associate_position	Associa ou move a posição de um ativo para uma carteira do usuário (editarInvestimento / associar ação à carteira).
remove_position	Remove todas as operações de um ativo do usuário e desfaz a associação com a carteira (removerInvestimento).

- **Interfaces fornecidas:**

Interface	Operações
IInvestmentService (Protocol Python, interfaces/components.py)	create_trade, list_trades, get_positions, associate_position, remove_position
API REST /api/stocks (routers/stocks.py)	POST /api/stocks, GET /api/stocks, GET /api/stocks/carteira?portfolio_id=, PUT /api/stocks/{ticker}/portfolio, DELETE /api/stocks/{ticker}

- **Interfaces Requeridas:**

Interface	Finalidade
IPortfolioService (Portfolio Component)	Validar a carteira informada (get_portfolio), associar e desassociar o ativo (associate_investment, dissociate_investment) e obter a carteira de cada ativo (get_investment_portfolios).
IMarketDataService (Market Data Component)	Validar se o ticker existe e está ativo na B3 (validate_ticker) e obter a cotação quando auto_cotacao = true (get_quote).
Session SQLAlchemy (PostgreSQL)	Persistir users_stocks e garantir as linhas de stocks e brokers.
get_current_user (Auth Component)	Identificar o usuário autenticado (JWT) nas rotas REST.

• Contratos:

Operação	Entradas	Saída	Pré-condição	Pós-condição
create_trade	user_id: int; trade_in: StockTradeCreate (ticker, quantidade > 0, preço > 0 ou auto_cotacao, broker_id, BUY/SELL, data com fuso, portfolio_id opcional)	UserStock (operação gravada)	Usuário autenticado. Com portfolio_id, a carteira é do usuário (senão 404). Ticker ativo na B3 (senão 404). Corretora existente (senão 400). Em venda, quantidade ≤ saldo (senão 422).	Operação gravada em users_stocks. Com portfolio_id, o ativo fica associado à carteira. Em erro de validação nada é gravado (rollback).
list_trades	user_id: int	list[UserStock]	Usuário autenticado.	Retorna só operações do usuário, mais recentes primeiro. Nenhum dado é alterado.
get_positions	user_id: int; portfolio_id: int None	list[PositionOut]	Se portfolio_id for informado, a carteira é do usuário (senão 404).	Uma posição por ativo com quantidade > 0, preço médio com 4 casas (vendas não alteram o preço médio) e carteira associada; filtrada pela carteira quando informada. Nenhum dado é alterado.
associate_position	user_id: int; stock_name: str; portfolio_id: int	PositionOut	O usuário tem posição aberta no ativo (senão 404). A carteira é do usuário (senão 404).	O ativo fica associado à carteira informada (é movido se estava em outra). As operações não mudam.
remove_position	user_id: int; stock_name: str	int (operações removidas)	Existe ao menos uma operação do ativo para o usuário (senão 404).	Todas as operações do ativo do usuário são removidas e a associação com a carteira é desfeita. Dados de outros usuários não mudam.

2. Componente 2

- **Nome:** Portfolio Component (backend/src/services/portfolio_service.py, classe PortfolioService)
- **Responsabilidade:** Gerenciar as carteiras do usuário (cadastrar, consultar, editar e remover) e manter a associação de cada ativo a uma carteira.
- **Operações:**

Operação	Descrição
create_portfolio	Cria uma carteira para o usuário (criarCarteira).
list_portfolios	Lista as carteiras do usuário em ordem alfabética (consultarCarteira).
get_portfolio	Consulta uma carteira pelo identificador, garantindo que pertence ao usuário (consultarCarteira).
update_portfolio	Altera nome e/ou descrição da carteira (editarCarteira).
delete_portfolio	Remove a carteira e suas associações, preservando as operações de compra e venda (removerCarteira).
associate_investment	Associa um ativo do usuário a uma carteira ou o move de carteira (associarInvestimento).
dissociate_investment	Desfaz a associação de um ativo com sua carteira (usada ao remover um investimento).
get_investment_portfolios	Retorna o mapa ativo → carteira do usuário (usada para montar as posições).

- **Interfaces fornecidas:**

Interface	Operações
IPortfolioService (Protocol Python, interfaces/components.py)	create_portfolio, list_portfolios, get_portfolio, update_portfolio, delete_portfolio, associate_investment, dissociate_investment, get_investment_portfolios
API REST /api/portfolios (routers/portfolios.py)	POST /api/portfolios, GET /api/portfolios, GET /api/portfolios/{id}, PATCH /api/portfolios/{id}, DELETE /api/portfolios/{id}

- **Interfaces Requeridas:**

Interface	Finalidade
Session SQLAlchemy (PostgreSQL)	Persistir as tabelas portfolios e portfolios_stocks.
get_current_user (Auth Component)	Identificar o usuário autenticado (JWT) nas rotas REST.

• Contratos:

Operação	Entradas	Saída	Pré-condição	Pós-condição
create_portfolio	user_id: int; portfolio_in: PortfolioCreate (nome com 1 a 100 caracteres, descrição opcional até 255)	Portfolio	Nome não vazio (senão 422). O usuário não tem carteira com o mesmo nome, sem diferenciar maiúsculas (senão 409).	Carteira gravada com portfolio_id e created_at, vinculada ao usuário.
list_portfolios	user_id: int	list[Portfolio]	Usuário autenticado.	Somente carteiras do usuário, em ordem alfabética. Nenhum dado é alterado.
get_portfolio	user_id: int; portfolio_id: int	Portfolio	A carteira existe e pertence ao usuário (senão 404).	Nenhum dado é alterado.
update_portfolio	user_id: int; portfolio_id: int; portfolio_in: PortfolioUpdate	Portfolio atualizada	Carteira do usuário (senão 404). Ao menos um campo enviado (senão 422). Novo nome livre (senão 409).	Somente os campos enviados mudam. As associações são preservadas.
delete_portfolio	user_id: int; portfolio_id: int	Nenhuma (None)	Carteira do usuário (senão 404).	Carteira e associações removidas. As operações de compra e venda são mantidas e os ativos ficam sem carteira.
associate_investment	user_id: int; portfolio_id: int; stock_id: int	Portfolio	Carteira do usuário (senão 404). stock_id existente (garantido pelo Investment Component).	O ativo fica associado a exatamente uma carteira (cria ou move a associação).
dissociate_investment	user_id: int; stock_id: int	bool	Nenhuma.	Associação removida. Retorna True se ela existia.
get_investment_portfolios	user_id: int	dict[stock_id, Portfolio]	Nenhuma.	Apenas associações do usuário. Nenhum dado é alterado.

3. Integração entre os componentes

A comunicação acontece dentro do mesmo processo, por interface. O `InvestmentService` recebe no construtor um objeto do tipo `IPortfolioService` e não importa a classe `PortfolioService`. A ligação é feita em `backend/src/dependencies/components.py`, pela injeção de dependências do FastAPI:

```
def get_investment_service(
    db: Session = Depends(get_db),
    market_data: IMarketDataService = Depends(get_market_data_service),
    portfolios: IPortfolioService = Depends(get_portfolio_service),
) -> IInvestmentService:
    return InvestmentService(db, market_data, portfolios)
```

Operação do Investment	Chamada a IPortfolioService	Finalidade
create_trade	get_portfolio e depois associate_investment	Validar a carteira antes de consultar a B3 e gravar; associar o ativo após gravar a operação.
get_positions	get_portfolio (com filtro) e get_investment_portfolios	Informar a carteira de cada posição e filtrar por carteira.
associate_position	associate_investment	Associar ou mover o ativo de carteira.
remove_position	dissociate_investment	Desfazer a associação do ativo removido.

Os erros de contrato do Portfolio (por exemplo, 404 para carteira inexistente ou de outro usuário) chegam sem alteração à resposta HTTP. Os testes de integração (`backend/tests/test_integration_investment_portfolio.py` e `test_api_integration.py`) verificam essa comunicação com os dois componentes reais.

4. Alterações em relação ao Lab 3 e justificativas

Alteração	Justificativa
O Investment Component também requer <code>IMarketDataService</code> , que não aparece no diagrama do Lab 3.	O cadastro de ação precisa validar se o ticker existe na B3 e, opcionalmente, buscar a cotação. Essa responsabilidade já é do Market Data Component; reutilizá-lo pela interface evita duplicar a integração com a <code>brapi.dev</code> .
<code>IPortfolioService</code> ganhou <code>get_portfolio</code> , <code>dissociate_investment</code> e <code>get_investment_portfolios</code> , além das cinco operações do Lab 3.	<code>consultarCarteira()</code> foi dividida em <code>listar</code> e <code>consultar por id</code> . As outras duas permitem ao Investment desfazer a associação ao remover um investimento e mostrar a carteira de cada posição sem acessar as tabelas do Portfolio diretamente.
<code>editarInvestimento()</code> foi implementada como troca de carteira (<code>associate_position</code>).	O preço médio é calculado a partir do histórico de operações, e a chave de <code>users_stocks</code> inclui a data da operação. Corrigir uma operação é feito removendo e registrando de novo, o que mantém o histórico consistente.



Alteração	Justificativa
Novas tabelas portfolios e portfolios_stocks (migração database/migrations/002_create_portfolios.sql).	O Portfolio precisa de dados próprios. A migração só cria tabelas; nenhuma tabela existente foi alterada, e o frontend continua funcionando (os campos novos das respostas são opcionais).
Operações nomeadas em inglês no código.	Segue o padrão do projeto. A correspondência com os nomes do Lab 3 está nas tabelas de operações e no README.